



Activation Functions

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

⚡ Activation Functions in Neural Networks

নিউরাল নেটওয়ার্কে Activation Function — সম্পূর্ণ বাংলা নোট

বিষয়: Non-linearity কেন দরকার? Sigmoid, ReLU, Tanh, এবং Softmax-এর বিস্তারিত ব্যাখ্যা।

পূর্বশর্ত: ANN Introduction, Forward Propagation সম্পর্কে ধারণা থাকতে হবে।

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একজন রান্নাঘরের সাহায্যকারী। তোমার কাজ হলো রান্নার উপকরণ (ingredients) গ্রহণ করা এবং রান্নাঘরের পরবর্তী ধাপে পাঠানো। কিন্তু একটা বিশেষ নিয়ম আছে — **তুমি সিদ্ধান্ত নিতে পারো** কোন উপকরণ এগিয়ে দেবে এবং কতটুকু দেবে।

এই "সিদ্ধান্ত নেওয়ার ক্ষমতা"ই হলো **Activation Function**।

বাস্তব জীবনের উদাহরণ:

🚦 ট্রাফিক লাইটের উদাহরণ:

- লাল আলো → গাড়ি থামো (output = 0)
- সবুজ আলো → গাড়ি চলো (output = 1)

- কিন্তু বাস্তব জীবনে সিদ্ধান্ত এত সহজ নয়! হলুদ আলোতে কী করবে? বৃষ্টিতে? রাস্তায় বাচ্চা থাকলে?

এই জটিল "বাস্তব" সিদ্ধান্তগুলো নেওয়ার জন্যই **Non-linear Activation Function** দরকার।

● মানুষের বায়োলজিক্যাল নিউরনের উদাহরণ:

- তুমি হাত পুড়িয়ে ফেললে → তাৎক্ষণিকভাবে হাত সরিয়ে নাও (threshold activation)
- তুমি ঠান্ডা জিনিস ধরলে → কোনো বিশেষ প্রতিক্রিয়া নেই
- মানুষের নিউরন শুধুমাত্র নির্দিষ্ট threshold অতিক্রম করলেই "fire" করে

এটাই Activation Function-এর মূল ধারণা!

এটি কোন সমস্যা সমাধান করে?

সমস্যা: যদি Activation Function না থাকে, তাহলে যত গভীর নেটওয়ার্কই বানাও না কেন, সবকিছু মিলিয়ে একটিমাত্র **Linear Transformation** হয়ে যাবে।

Layer 1: $y = W1 * x + b1$

Layer 2: $y = W2 * (W1 * x + b1) + b2$

$= (W2*W1)*x + (W2*b1 + b2)$

$= W_new * x + b_new$ ← এটা আবার একটা Linear Equation!

সমাধান: Activation Function দিয়ে **Non-linearity** যোগ করে, তাহলে নেটওয়ার্ক জটিল patterns শিখতে পারবে।

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

Non-linearity কেন এত গুরুত্বপূর্ণ?

একটা খুব সহজ উদাহরণ দিয়ে বুঝি। তুমি কি ছবিতে **বিড়াল** চেনাতে পারবে শুধুমাত্র একটি সরলরেখা দিয়ে?

❌ Linear সিদ্ধান্ত সীমা:

বিড়াল | কুকুর



(শুধু সরলরেখায় ভাগ করা যায়)

✅ Non-linear সিদ্ধান্ত সীমা:



(বাঁকা, জটিল boundaries)

বাস্তব ডেটা (ছবি, ভয়েস, টেক্সট) কখনো সরলরেখায় বিভক্ত হয় না। তাই Non-linearity অপরিহার্য।

কিভাবে Activation Function কাজ করে?

প্রতিটি নিউরনে দুটো ধাপ হয়:

ধাপ ১: $z = (w_1 \times x_1) + (w_2 \times x_2) + \dots + \text{bias}$ ← Linear combination

ধাপ ২: $a = f(z)$ ← Activation Function প্রয়োগ

এখানে $f()$ হলো Activation Function যা z -কে একটি অর্থবহ output-এ রূপান্তরিত করে।

চারটি প্রধান Activation Function:

১. Sigmoid Function

স্বাভাবিক ভাষায়: সব ইনপুটকে ০ থেকে ১-এর মধ্যে "চাপ দিয়ে" ঢুকিয়ে দেয়। অনেকটা probability-র মতো আচরণ করে।

কীভাবে কাজ করে:

- খুব বড় positive সংখ্যা → ১ এর কাছে যায়
- খুব বড় negative সংখ্যা → ০ এর কাছে যায়
- ০ ইনপুট → ঠিক ০.৫ আউটপুট

২. ReLU (Rectified Linear Unit)

স্বাভাবিক ভাষায়: একটি সহজ নিয়ম — যদি ইনপুট নেগেটিভ হয়, ০ দাও; যদি পজিটিভ হয়, সেটাই দাও।

কীভাবে কাজ করে:

- নেগেটিভ ইনপুট $\rightarrow 0$
- পজিটিভ ইনপুট $\rightarrow$ ইনপুট যা আছে তাই

এটা একটা "স্বেচ্ছাচারী ফিল্টার" যা শুধু পজিটিভ সংকেত পাঠায়।

৩. Tanh (Hyperbolic Tangent)

স্বাভাবিক ভাষায়: Sigmoid-এর মতোই, কিন্তু -1 থেকে $+1$ এর মধ্যে। Zero-centered মানে নেগেটিভ মানও সম্ভব।

কীভাবে কাজ করে:

- Sigmoid-এর উন্নত সংস্করণ
- নেগেটিভ ইনপুট $\rightarrow$ নেগেটিভ আউটপুট (-1 এর দিকে)
- পজিটিভ ইনপুট $\rightarrow$ পজিটিভ আউটপুট ($+1$ এর দিকে)

৪. Softmax Function

স্বাভাবিক ভাষায়: একাধিক class-এর জন্য probability distribution তৈরি করে। সব probability-র যোগফল সবসময় ১।

কীভাবে কাজ করে:

- ৩টি class থাকলে $\rightarrow$ ৩টি probability বের করে
- সবগুলোর যোগফল = ১.০
- সবচেয়ে বড় probability = সবচেয়ে সম্ভাব্য class

৩. Math / Theory

● Sigmoid Function:

সূত্র:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

প্রতিটি symbol-এর অর্থ:

- σ (sigma) = Sigmoid function-এর নাম
- z = ইনপুট মান (যেকোনো real number)
- e = Euler's number ≈ 2.718

- $e^{(-z)}$ = e এর (-z) power

Derivative (Gradient):

$$\sigma'(z) = \sigma(z) \times (1 - \sigma(z))$$

Manual Calculation (উদাহরণ):

$z = 2$ হলে:

$$\begin{aligned}\sigma(2) &= 1 / (1 + e^{(-2)}) \\ &= 1 / (1 + 0.135) \\ &= 1 / 1.135 \\ &= 0.880\end{aligned}$$

$z = 0$ হলে:

$$\begin{aligned}\sigma(0) &= 1 / (1 + e^0) \\ &= 1 / (1 + 1) \\ &= 1 / 2 \\ &= 0.5\end{aligned}$$

$z = -3$ হলে:

$$\begin{aligned}\sigma(-3) &= 1 / (1 + e^3) \\ &= 1 / (1 + 20.09) \\ &= 1 / 21.09 \\ &= 0.047\end{aligned}$$

Range: (0, 1) — কখনো ঠিক 0 বা 1 হয় না, শুধু কাছাকাছি যায়।

 ReLU Function:

সূত্র:

$$f(z) = \max(0, z)$$

অর্থাৎ:

$$z, \text{ যদি } z > 0$$

$$f(z) =$$

$$0, \text{ যদি } z \leq 0$$

Derivative:

$$1, \text{ যদি } z > 0$$

$$f'(z) =$$

$$0, \text{ যদি } z < 0$$

($z = 0$ তে undefined, কিন্তু সাধারণত 0 ধরা হয়)

Manual Calculation:

$$z = 5.3 \rightarrow f(5.3) = \max(0, 5.3) = 5.3$$

$$z = -2.7 \rightarrow f(-2.7) = \max(0, -2.7) = 0$$

$$z = 0 \rightarrow f(0) = \max(0, 0) = 0$$

$$z = 100 \rightarrow f(100) = \max(0, 100) = 100$$

Range: $[0, \infty)$ — শূন্য থেকে অসীম পর্যন্ত।

● Tanh Function:

সূত্র:

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

অথবা Sigmoid দিয়ে:

$$\tanh(z) = 2\sigma(2z) - 1$$

Derivative:

$$\tanh'(z) = 1 - \tanh^2(z)$$

Manual Calculation:

$z = 1$ হলে:

$$\begin{aligned}\tanh(1) &= (e^1 - e^{-1}) / (e^1 + e^{-1}) \\ &= (2.718 - 0.368) / (2.718 + 0.368) \\ &= 2.350 / 3.086 \\ &= 0.762\end{aligned}$$

$z = 0$ হলে:

$$\tanh(0) = (1 - 1) / (1 + 1) = 0/2 = 0$$

$z = -1$ হলে:

$$\tanh(-1) = -0.762 \leftarrow \text{Sigmoid-এর মতো নয়, নেগেটিভ হতে পারে!}$$

Range: $(-1, 1)$ — শূন্যকেন্দ্রিক (zero-centered)।

● Softmax Function:

সূত্র:

$$\text{Softmax}(z_i) = e^{(z_i)} / \sum_j e^{(z_j)}$$

প্রতিটি symbol:

- z_i = i -তম class-এর raw score (logit)
- $e^{(z_i)}$ = i -তম class-এর exponential score
- $\sum_j e^{(z_j)}$ = সব class-এর exponential score-এর যোগফল

Manual Calculation (৩টি class):

Raw scores (z): [2.0, 1.0, 0.1] ← বিড়াল, কুকুর, পাখি

Step 1: Exponential নাও

$$e^{2.0} = 7.39$$

$$e^{1.0} = 2.72$$

$$e^{0.1} = 1.11$$

Step 2: যোগফল করো

$$\text{Sum} = 7.39 + 2.72 + 1.11 = 11.22$$

Step 3: Normalize করো

$$\text{Softmax}(\text{বিড়াল}) = 7.39 / 11.22 = 0.659 \text{ (65.9\%)}$$

$$\text{Softmax}(\text{কুকুর}) = 2.72 / 11.22 = 0.242 \text{ (24.2\%)}$$

$$\text{Softmax}(\text{পাখি}) = 1.11 / 11.22 = 0.099 \text{ (9.9\%)}$$

$$\text{যাচাই: } 0.659 + 0.242 + 0.099 = 1.000 \quad \checkmark$$

8. Code Example (Python)


```

import numpy as np
import matplotlib.pyplot as plt

# =====
# ১. Sigmoid Function
# =====

def sigmoid(z):
    """Sigmoid: যেকোনো সংখ্যাকে (0, 1) এর মধ্যে নিয়ে আসে"""
    return 1 / (1 + np.exp(-z)) # সূত্র প্রয়োগ

def sigmoid_derivative(z):
    """Sigmoid-এর derivative – Backpropagation-এ দরকার"""
    s = sigmoid(z)
    return s * (1 - s) #  $\sigma(z) \times (1 - \sigma(z))$ 

# পরীক্ষা করো
test_values = [-5, -2, -1, 0, 1, 2, 5] # বিভিন্ন ইনপুট
print("=== Sigmoid ===")
for z in test_values:
    output = sigmoid(z)
    print(f" sigmoid({z:3d}) = {output:.4f}") # ৪ দশমিক ঘর

# আউটপুট:
# === Sigmoid ===
# sigmoid( -5) = 0.0067
# sigmoid( -2) = 0.1192
# sigmoid( -1) = 0.2689
# sigmoid( 0) = 0.5000
# sigmoid( 1) = 0.7311
# sigmoid( 2) = 0.8808
# sigmoid( 5) = 0.9933

# =====
# ২. ReLU Function
# =====

def relu(z):
    """ReLU: নেগেটিভ হলে ০, পজিটিভ হলে সেই মান"""
    return np.maximum(0, z) # max(0, z) – NumPy দিয়ে

```

```

def relu_derivative(z):
    """ReLU-এর derivative: 1 যদি  $z > 0$ , নয়তো 0"""
    return np.where(z > 0, 1, 0) # condition-based return

# পরীক্ষা করো
print("\n=== ReLU ===")
for z in test_values:
    output = relu(z)
    print(f"  relu({z:3d}) = {output:.4f}")

# আউটপুট:
# === ReLU ===
#   relu( -5) = 0.0000
#   relu( -2) = 0.0000
#   relu( -1) = 0.0000
#   relu(  0) = 0.0000
#   relu(  1) = 1.0000
#   relu(  2) = 2.0000
#   relu(  5) = 5.0000

# =====
# ৩. Tanh Function
# =====

def tanh_custom(z):
    """Tanh: যেকোনো সংখ্যাকে (-1, 1) এর মধ্যে নিয়ে আসে"""
    return np.tanh(z) # NumPy-র built-in tanh

def tanh_derivative(z):
    """Tanh-এর derivative:  $1 - \tanh^2(z)$ """
    return 1 - np.tanh(z)**2

# পরীক্ষা করো
print("\n=== Tanh ===")
for z in test_values:
    output = tanh_custom(z)
    print(f"  tanh({z:3d}) = {output:.4f}")

# আউটপুট:
# === Tanh ===
#   tanh( -5) = -1.0000
#   tanh( -2) = -0.9640

```

```

# tanh( -1) = -0.7616
# tanh(  0) = 0.0000
# tanh(  1) = 0.7616
# tanh(  2) = 0.9640
# tanh(  5) = 1.0000

# =====
# 8. Softmax Function
# =====

def softmax(z):
    """
    Softmax: Multi-class probability distribution তৈরি করে
    Numerically stable version ব্যবহার করা হয়েছে
    """
    # Numerical stability-র জন্য max বিয়োগ করো (overflow রোধ)
    e_z = np.exp(z - np.max(z)) # প্রতিটি থেকে max বিয়োগ
    return e_z / e_z.sum()      # normalize করো

# পরীক্ষা করো – ৩টি class (বিড়াল, কুকুর, পাখি)
raw_scores = np.array([2.0, 1.0, 0.1]) # model-এর raw output
probabilities = softmax(raw_scores)    # probability-তে রূপান্তর

print("\n=== Softmax ===")
classes = ["বিড়াল", "কুকুর", "পাখি"] # ক্লাসের নাম
for i, (cls, prob) in enumerate(zip(classes, probabilities)):
    print(f" {cls}: {prob:.4f} ({prob*100:.1f}%)")
print(f" যোগফল: {probabilities.sum():.4f}") # সবসময় 1.0

# আউটপুট:
# === Softmax ===
# বিড়াল: 0.6590 (65.9%)
# কুকুর: 0.2424 (24.2%)
# পাখি: 0.0986 (9.9%)
# যোগফল: 1.0000

# =====
# ৫. Neural Network-এ Activation Function ব্যবহার
# =====

class SimpleNeuralNetwork:

```

```

"""একটি সহজ Neural Network – activation functions সহ"""

def __init__(self):
    np.random.seed(42) # reproducibility-র জন্য
    # Layer weights এবং biases initialize করো
    self.W1 = np.random.randn(3, 4) * 0.1 # Input → Hidden
    self.b1 = np.zeros((1, 4)) # Hidden layer bias
    self.W2 = np.random.randn(4, 2) * 0.1 # Hidden → Output
    self.b2 = np.zeros((1, 2)) # Output layer bias

def forward(self, X):
    """Forward pass: ReLU hidden layer, Softmax output"""
    # Hidden layer – ReLU activation
    self.z1 = np.dot(X, self.W1) + self.b1 # Linear combination
    self.a1 = relu(self.z1) # ReLU প্রয়োগ

    # Output layer – Softmax activation
    self.z2 = np.dot(self.a1, self.W2) + self.b2 # Linear combination
    # প্রতিটি sample-এ আলাদা softmax
    self.a2 = np.array([softmax(row) for row in self.z2])

    return self.a2

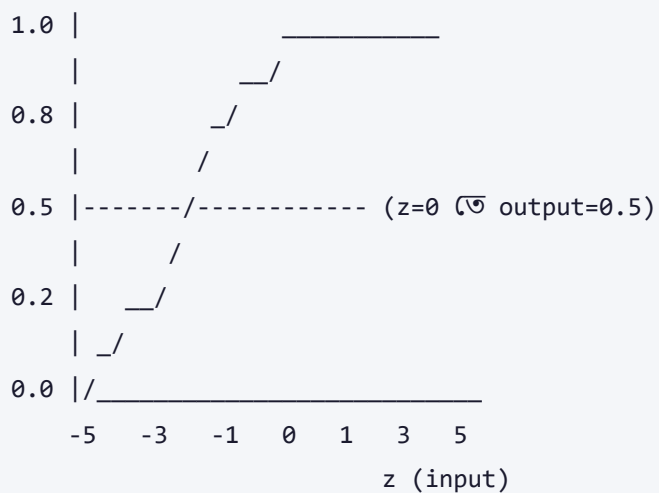
# পরীক্ষা করো
nn = SimpleNeuralNetwork()
X_test = np.array([[0.5, 0.3, 0.8]]) # একটি sample input
output = nn.forward(X_test) # forward pass চালাও
print(f"\n=== Neural Network Output ===")
print(f" Input: {X_test[0]}")
print(f" Class 0 Probability: {output[0][0]:.4f}")
print(f" Class 1 Probability: {output[0][1]:.4f}")

```

🔗 🎨 Visual / Diagram

Sigmoid — S-আকৃতির curve:

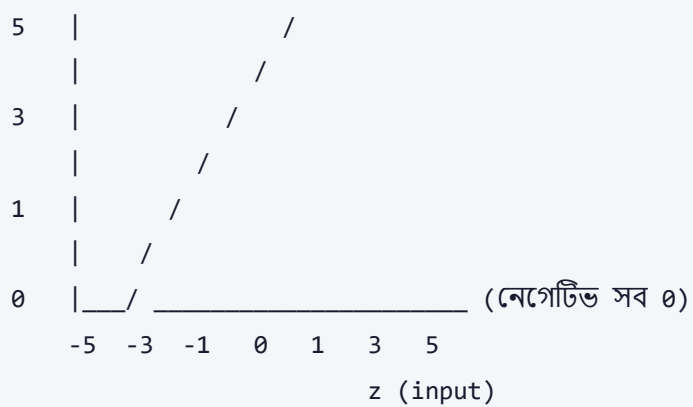
output



↑ S-আকৃতি ↑ কখনো 0 বা 1 স্পর্শ করে না

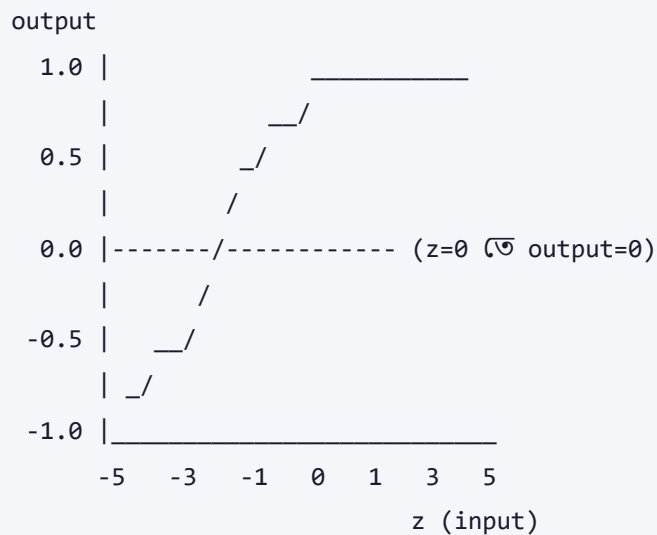
ReLU — Simple কিন্তু কার্যকর:

output



↑ নেগেটিভ কাটো, পজিটিভ রাখো ↑

Tanh — Zero-centered S-curve:



↑ Sigmoid-এর মতো কিন্তু -1 থেকে +1 ↑

Softmax — Probability Distribution:

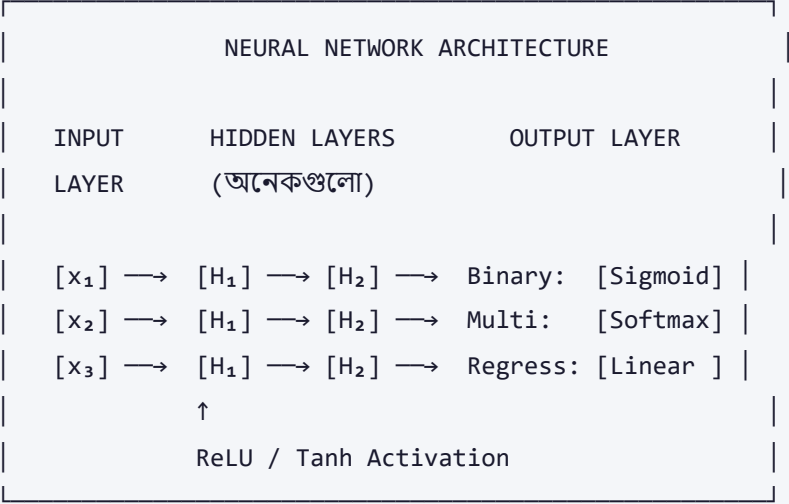
Raw Scores (Logits) Softmax Output (Probabilities)

বিড়াল: 2.0	→	বিড়াল: ██████████ 65.9%
কুকুর: 1.0	→	কুকুর: █████ 24.2%
পাখি: 0.1	→	পাখি: █ 9.9%

↓ Softmax

যোগফল = 100% ✓

Neural Network-এ কোথায় কোন Activation Function:



Hidden Layers: ReLU (default) অথবা Tanh (RNN-এ)

Output Layer: কাজ অনুযায়ী ভিন্ন

তুলনামূলক চিত্র:

Function	Input=-5	Input=0	Input=5	Range
Sigmoid	0.007	0.500	0.993	(0, 1)
ReLU	0.000	0.000	5.000	[0, ∞)
Tanh	-1.000	0.000	1.000	(-1, 1)
Softmax	(vector input → probability distribution)			

৬. Real-world Use Cases

১. Image Classification (ReLU + Softmax)

Google Photos, Instagram, Facebook

- CNN-এর hidden layers-এ ReLU ব্যবহার
- আউটপুটে Softmax দিয়ে বিড়াল/কুকুর/মানুষ ইত্যাদি নির্ধারণ
- উদাহরণ: ResNet, VGG, EfficientNet মডেল

২. 📧 Email Spam Detection (Sigmoid)

Gmail-এর Spam Filter

- Binary classification: স্প্যাম (১) নাকি নয় (০)?
- আউটপুট লেয়ারে Sigmoid ব্যবহার
- Output ০.৫-এর বেশি হলে spam, কম হলে নয়

৩. 🗣️ Language Translation (Tanh + Softmax)

Google Translate, DeepL

- RNN/LSTM-এর hidden state আপডেটে Tanh ব্যবহার
- প্রতিটি শব্দ নির্বাচনে Softmax দিয়ে vocabulary-র উপর probability বণ্টন
- উদাহরণ: seq2seq মডেল

৪. 🎮 Game AI (ReLU)

AlphaGo, OpenAI Five

- Deep Reinforcement Learning-এ ReLU hidden layers
- দ্রুত computation-এর জন্য ReLU আদর্শ
- লক্ষ লক্ষ parameter দিয়ে জটিল game strategy শেখা

৫. 🏥 Medical Diagnosis (Sigmoid/Softmax)

X-ray/MRI বিশ্লেষণ

- **Sigmoid:** রোগ আছে কি নেই? (binary)
- **Softmax:** রোগটি কোন ধরনের? (multi-class)
- IBM Watson Health, Google Health-এর মতো platform ব্যবহার করে

৭. ⚖️ Pros & Cons

Sigmoid:

সুবিধা ✓	অসুবিধা ✗
Output (0,1) → probability হিসেবে ব্যাখ্যা সহজ	Vanishing Gradient সমস্যা (গভীর নেটওয়ার্কে ভয়াবহ)
মসৃণ ও differentiable সব জায়গায়	Zero-centered নয় (optimization কঠিন হয়)
Binary classification-এ উপযুক্ত	Computationally costly (e^x হিসাব করতে হয়)
Output সবসময় bounded	Saturates quickly (gradient = 0 হয়ে যায়)

ReLU:

সুবিধা ✓	অসুবিধা ✗
Computationally খুব সহজ ও দ্রুত	Dying ReLU: নেগেটিভ input পেলে নিউরন "মরে যায়"
Vanishing Gradient অনেক কম	Zero-centered নয়
Deep network-এ ভালো কাজ করে	নেগেটিভ input-এ gradient = 0 (learning বন্ধ)
Sparse activation: efficient	Unbounded output (exploding gradient সম্ভব)

Tanh:

সুবিধা ✓	অসুবিধা ✗
Zero-centered (-1 থেকে 1) → বেটার optimization	Vanishing Gradient (Sigmoid-এর চেয়ে কম কিন্তু আছে)
Sigmoid-এর চেয়ে শক্তিশালী gradient	Sigmoid-এর মতো saturate হয়
RNN/LSTM-এ ভালো কাজ করে	Computationally costly
নেগেটিভ আউটপুট সম্ভব	গভীর নেটওয়ার্কে সমস্যা

Softmax:

সুবিধা ✓	অসুবিধা ✗
Multi-class probability নিশ্চিত করে	শুধু output layer-এ ব্যবহারযোগ্য
সব output-এর যোগফল = 1 (interpretable)	Large number-এ numerical instability হতে পারে
Cross-entropy loss-এর সাথে উপযুক্ত	Class সংখ্যা বাড়লে computation বাড়ে
ক্লাসগুলোর মধ্যে তুলনা দেখায়	Mutually exclusive class ধরে নেয়

৮. ⚠ Common Mistakes & Gotchas

ভুল ১: Hidden Layer-এ Softmax ব্যবহার

✗ ভুল – Hidden layer-এ Softmax দিলে সব নিউরন একসাথে compete করে

```
hidden = softmax(np.dot(X, W1) + b1)
```

✓ সঠিক – Hidden layer-এ ReLU ব্যবহার করো

```
hidden = relu(np.dot(X, W1) + b1)
```

Output layer-এ Softmax

```
output = softmax(np.dot(hidden, W2) + b2)
```

ভুল ২: Deep Network-এ Sigmoid বা Tanh (Vanishing Gradient)

Layer 1 Gradient: 0.25

Layer 2 Gradient: $0.25 \times 0.25 = 0.0625$

Layer 3 Gradient: $0.0625 \times 0.25 = 0.0156$

Layer 10 Gradient: ≈ 0.000001 ← প্রায় শূন্য!

Network শিখতে পারছে না!

ভুল ৩: Softmax-এ Numerical Overflow

```
# ❌ ভুল – বড় সংখ্যায় overflow হবে
def bad_softmax(z):
    return np.exp(z) / np.sum(np.exp(z))
# z = [1000, 999, 998] → np.exp(1000) = inf !

# ✅ সঠিক – max বিয়োগ করে numerical stability নিশ্চিত করে
def good_softmax(z):
    e_z = np.exp(z - np.max(z)) # overflow রোধ
    return e_z / e_z.sum()
```

ভুল ৪: Dying ReLU সমস্যা

যখন: Learning rate বেশি, negative weight initialization
 ফলাফল: অনেক নিউরনের input সবসময় নেগেটিভ

- ReLU সবসময় ০ দেয়
- Gradient = ০
- Weights আর update হয় না!
- Neuron "died" (মরে গেছে)

সমাধান: Leaky ReLU বা He Initialization ব্যবহার করো

ভুল ৫: Binary Classifier-এ Softmax ব্যবহার

```
# ❌ ভুল – Binary classification-এ Softmax অতিরিক্ত
output = softmax(z) # 2 class-এর জন্য

# ✅ সঠিক – একটি Sigmoid যথেষ্ট
output = sigmoid(z) # শুধু একটি output, threshold 0.5
```

ভুল ৬: Output Layer-এ ReLU ব্যবহার

❌ ভুল – Classification-এ ReLU output দিলে probability পাওয়া যাবে না
 $\text{output} = \text{relu}(z)$ # 0 বা positive কিছু $\rightarrow$ interpretation কঠিন

✅ সঠিক – classification task-এ Sigmoid বা Softmax
 $\text{output} = \text{sigmoid}(z)$ # binary
 $\text{output} = \text{softmax}(z)$ # multi-class

৯. 📌 Related Topics

আগে কী জানা দরকার?

1. **ANN Introduction** — Perceptron, Forward Propagation
2. **Linear Algebra** — Matrix multiplication, dot product
3. **Calculus** — Derivative, Chain rule (gradient-এর জন্য)
4. **Sigmoid Function** — Logistic Regression-এ আগেই দেখেছে

পরে কী শেখা উচিত?

1. **Backpropagation** — এই activation functions-এর derivative কীভাবে ব্যবহার হয়
2. **Vanishing/Exploding Gradient** — কেন গভীর নেটওয়ার্কে সমস্যা হয়
3. **Advanced Activation Functions:**

- **Leaky ReLU:** $f(z) = \max(0.01z, z)$ — Dying ReLU সমাধান
- **ELU:** Negative range-এ smooth curve
- **GELU:** Transformer-এ ব্যবহৃত (BERT, GPT)
- **Swish:** Google Brain-এর আবিষ্কার: $f(z) = z \times \text{sigmoid}(z)$

1. **Loss Functions** — Cross-Entropy (Sigmoid/Softmax-এর সাথে)
2. **Optimizers** — SGD, Adam (gradients ব্যবহার করে weight update)

Activation Function $\rightarrow$ Task Mapping:

Task Type	→ Recommended Output Activation
Binary Classification	→ Sigmoid
Multi-class (Exclusive)	→ Softmax
Regression	→ None (Linear)
Hidden Layers	→ ReLU (default)
RNN Hidden State	→ Tanh

30. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল:

S-R-T-S = "Start Right There, Sir!"

- **Sigmoid** → ০ থেকে ১ (probability-র মতো, binary classification)
- **ReLU** → সহজ: নেগেটিভ হলে শূন্য, পজিটিভ হলে সেটাই রাখো
- **Tanh** → Sigmoid-এর মতো কিন্তু -১ থেকে +১ (zero-centered)
- **Softmax** → সব probability যোগ করলে ১ (multi-class)

এক লাইনে সারসংক্ষেপ:

Function	১ লাইনে	ব্যবহার
Sigmoid	"চাপ দিয়ে ০-১-এ ঢোকাও"	Binary আউটপুট
ReLU	"নেগেটিভ কাটো, পজিটিভ রাখো"	Hidden layers (ডিফল্ট)
Tanh	"Sigmoid-এর ভাই, কিন্তু -১ থেকে +১"	RNN hidden state
Softmax	"সবাই মিলে ১০০% ভাগ করো"	Multi-class আউটপুট

Visual Memory Aid:

🌀 কোন Layer-এ কী?

Hidden Layers →	● ReLU (দ্রুত, কার্যকর)	
	● Tanh (RNN-এ)	
Output Layer →	● Sigmoid (binary: হ্যাঁ/না)	
	● Softmax (multi: কোনটা?)	
	● Linear (regression)	

Vanishing Gradient মনে রাখার ট্রিক:

গভীর → Sigmoid/Tanh → Gradient ছোট হয় → শেখা বন্ধ
 ↑
 এটা এড়াতে ReLU ব্যবহার!

চূড়ান্ত ১ লাইন:

"Activation Function হলো নিউরনের সিদ্ধান্ত নেওয়ার ক্ষমতা — এটা ছাড়া Neural Network শুধুই একটি ব্যর্থ সরলরেখা!"

📅 নোট তৈরির তারিখ: ২০২৬-০৪-০৪ | 🤖 AI-assisted Bengali ML Notes

📖 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

🤖 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes